

a breakpoint, in the *Editor* window, double-click on the line number. Once a breakpoint is set, a small circle will be shown at that line. For instance, Figure 6 shows that breakpoint is set at line no. 9.

To start the debugger, select *Class* from the *Package Explorer*->*right click*->*Debug As*->*Java Application*. It will open the debugger window from which you can inspect variables.

In addition, this perspective provides a toolbar that allows you to resume execution, terminate debugging, step into a method, step over, step return and so on. For instance, Figure 8 shows that, after performing the 'step into' action, we can inspect the value of the variable 'c'.

Power actions

Eclipse is a feature-rich IDE that provides many power actions which make a programmer's life easier. In this section, we'll look at some power actions like code generation, code formatting and so on.

- 1) **Code generation:** Eclipse allows us to generate code for the getter/setters, constructors, toString, hashCode and equals methods. To generate code, right click in the *Editor* window, select the source and choose the option you need.
- 2) **Code refactoring:** Eclipse allows us to perform refactoring at the project, package or class levels. To perform refactoring, select *Project/Package/Class* from the 'Package Explorer', right click on it and select the 'Refactor' option. It will allow various actions like *rename*, *move*, and so on.
- 3) **Code formatting:** Eclipse allows us to provide code formatting actions. Using this we can instruct Eclipse to remove unwanted spaces, indent code according to our choice, and so on. Let us unindent code deliberately, as follows:

```
package com.osfy;

public class AdditionExample {
```

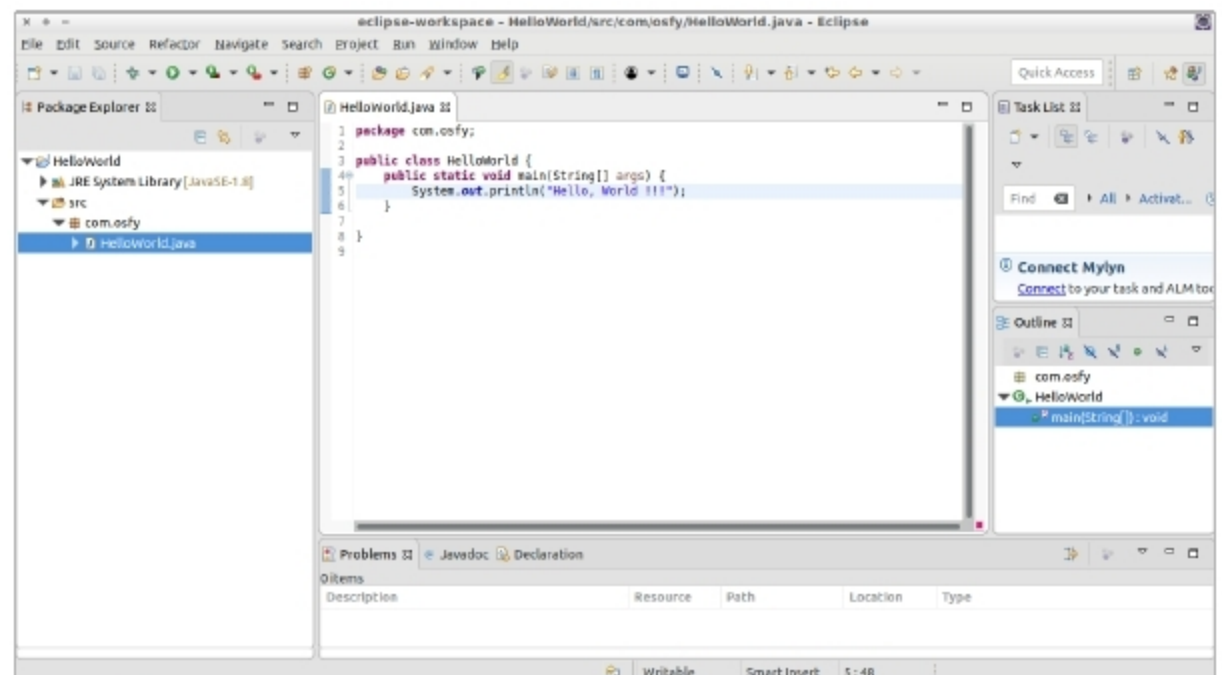


Figure 4: Eclipse class

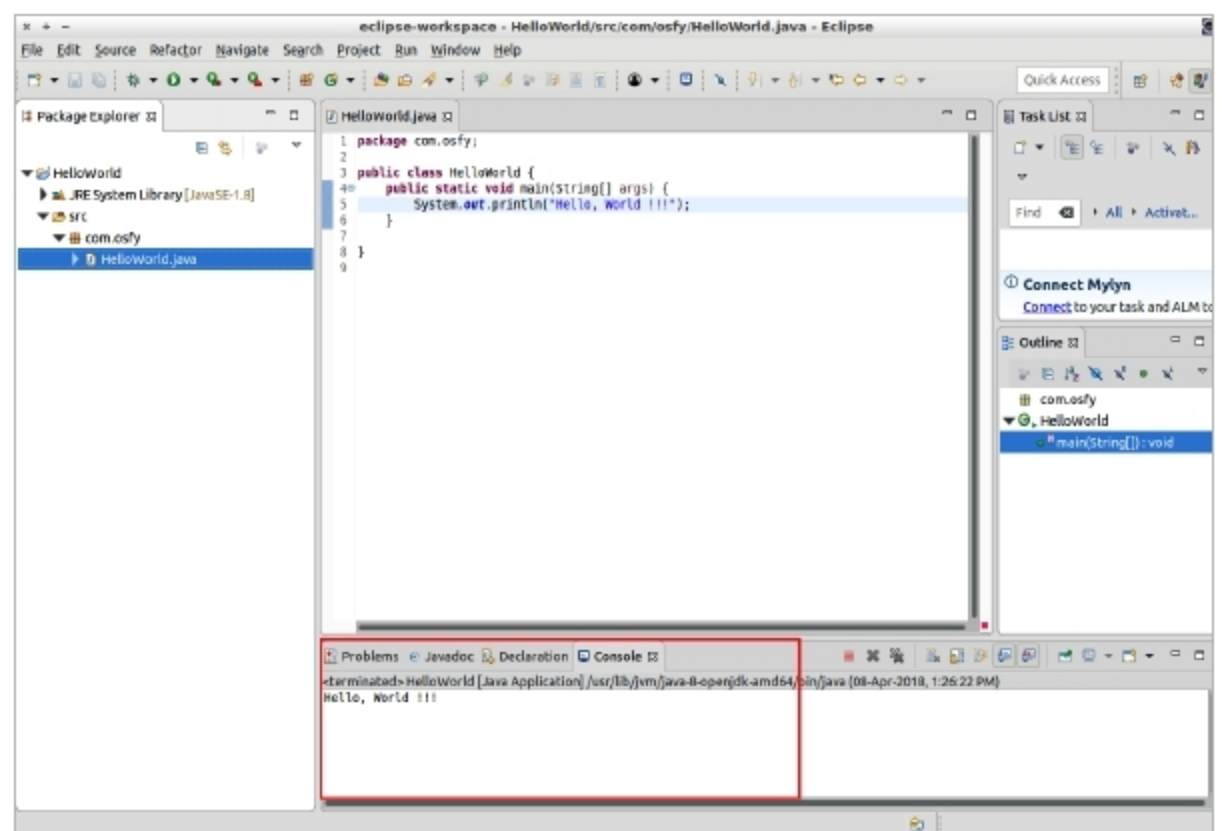


Figure 5: Eclipse output

```
public static void
main(String[] args) {
    int a, b, c; a = 10; b =
20; c = a + b;
    System.out.println(a + " + " +
b + " = " + c);
}
```

Now, to format this code, select the code block ->*Right click*->*Source*->*Format*. This will auto-indent the code.

- 4) **Organise imports:** Java provides a rich set of collections. Each collection is defined in a separate

class/package. Remembering each class/package's name will be overwhelming and here Eclipse provides a solution.

To understand this, let us add the following new line into our Java class:

```
List<Integer> = new ArrayList<>();
```

To work this program correctly, we have to import the *java.util.ArrayList* and *java.util.List* packages into the current program. Obviously, we can do this manually but why bother when Eclipse can do it for you.

Now to organise imports, right click in the *Editor window*->*Source*-